

Naive Bayes:

generative model.

Prior:

dataset with n samples.

k_1 examples with class 1.

k_2 " " class 2.

$$k_1 + k_2 = n.$$

$$P(C_1) = \frac{k_1}{n}$$

$$P(C_2) = \frac{k_2}{n}$$

prior probability,

$$P(\text{ham}) = 0.8$$

$$P(\text{spam}) = 0.2$$

given that 80% of samples in training is ham.

"free prize today" $\rightarrow$ spam.

$P(\text{free}|\text{spam}) \Rightarrow$ Likelihood of word free.

Training { $P(\text{spam}) \rightarrow$ prior $\rightarrow$ $\frac{\text{no. of spam examples}}{\text{total no. of examples}}$
 $P(\text{free}|\text{spam}) \rightarrow$ likelihood.

Inference { $P(\text{spam}|\text{free}) \rightarrow$ prediction.

$$P(c|d) = \frac{P(d|c) \cdot P(c)}{P(d)}$$

document / message / word.

Inference, $\{ P(c|d) \Rightarrow \text{Posterior} \Rightarrow \text{belief about this class } c \text{ after seeing doc. } d.$

Training, $\{ \begin{aligned} P(c) &\Rightarrow \text{prior} \\ P(d|c) &\Rightarrow \text{Likelihood} \\ P(d) &\Rightarrow \text{Evidence,} \end{aligned}$

$$P(c|d) = \underbrace{P(d|c)}_{\text{Likelihood}} \cdot \underbrace{P(c)}_{\text{Prior}}$$

$$\hat{c} = \underset{c}{\operatorname{argmax}} P(c|d)$$

$$\hat{c} = \underset{c}{\operatorname{argmax}} P(d|c) \cdot P(c)$$

Dataset:

Tokenization:

free prize claim wow $\rightarrow$ [free, prize, claim, wow]

vocabulary:

$$d = [w_1, w_2, w_3, \dots, w_v]$$

where v is total no. of unique words

Bag-of-words.

message	free	prize	meeting	today	project	claim
free prize claim	1	1	0	0	0	1
project meeting today	0	0	1	1	1	0
free free prize	2	1	0	0	0	0

Class-wise word counts:

word	count in ham	count in spam
free	0	4
claim	0	3
meeting	4	0
project	3	0
today	2	1

$$P(\text{free} | \text{spam}) = \frac{\text{count of word free in spam}}{\text{count of total words in spam}}$$

$$= \frac{4}{30} \rightarrow \text{assuming,}$$

$$P(w|c) = \frac{\text{count of word } w \text{ in class } c}{\text{count of total words in class } c}$$

Naive Bayes assumes that all words are independent.

$$P(w_1, w_2, w_3 \dots, w_n | c) = P(w_1 | c) \cdot P(w_2 | c) \cdot \dots \cdot P(w_n | c) \\ = \prod_{i=1}^n P(w_i | c)$$

$$\hat{c} = \underset{c}{\operatorname{argmax}} \prod_{i=1}^n P(w_i | c) \cdot P(c)$$

Free prize today.

class = ham.

$$P(\text{free} | \text{ham}) \times P(\text{prize} | \text{ham}) \times P(\text{today} | \text{ham}) \times P(\text{ham})$$

class = spam

$$P(\text{free} | \text{spam}) \times P(\text{prize} | \text{spam}) \times P(\text{today} | \text{spam}) \times P(\text{spam})$$

Laplace

Smoothing: (to avoid being prob 0 for a word)

$$P(w | c) = \frac{\text{count}(w, c) + \alpha}{\sum_{w \in V} \text{count}(w, c) + \alpha |V|}$$

if $\alpha = 1$,

$$P(w|c) = \frac{\text{count}(w, c) + 1}{\sum_{w \in V} \text{count}(w, c) + |V|}$$

if $\text{count}(w, c) = 0$

↳ no. of words in
vocabulary

$$P(w, c) = \frac{0 + 1}{30 + 60} = \frac{1}{90}$$

$$\log ab = \log a + \log b$$

$$\log \left(\prod_{i=1}^n P(w_i | c) \cdot P(c) \right) =$$

$$\sum_{i=1}^n \log P(w_i | c) + \log P(c)$$

Full training ALSO

- ① compute Prior $P(c)$
- ② build the vocab, dataset.
- ③ compute ^{count} words in each class.
- ④ compute likelihood with smoothing.
- ⑤ inference $\rightarrow P(c|d)$

Inference:

- ① tokenize.
- ② $P(w|c)$ for each word in each class

③ Return the class with highest score.

Variants.

① multinomial NB: take exact count

② Bernoulli NB: features are binary,

③ Gaussian NB:

$$P(x_j | c) = \frac{1}{\sqrt{2\pi\sigma_{c_j}^2}} e^{\left(\frac{-(x_j - \mu_{c_j})^2}{2\sigma_{c_j}^2}\right)}$$

